

Grado en Ingeniería Electrónica de Comunicaciones
Procesamiento de Señales · Curso 2026-2027

Apuntes

Tema 5. Aplicaciones

Jesús Chacón Sombría

Cómo usar estos apuntes

Estos apuntes desarrollan el contenido de las tres presentaciones del tema: la sección 1 corresponde a la parte 1 (transformada de Fourier de tiempo corto y espectrograma), la sección 2 a la parte 2 (wavelets) y la sección 3 a la parte 3 (procesamiento de imágenes). Los ejemplos de código están en `teoría/T5/codigo/`, en Python (`numpy` y `scipy`) y en Matlab, y se pueden ejecutar tal cual; ninguno necesita toolboxes ni ficheros externos. Al final de cada sección se indican los ejercicios de la hoja del tema que la practican.

Índice general

1 Transformada de Fourier de tiempo corto 2
1.1 Señales no estacionarias . . . 2
1.2 Definición . . . 3
1.3 Espectrograma . . . 4
1.4 Resolución en tiempo y en frecuencia . . . 4
1.5 Síntesis por solapamiento y suma . . . 5
1.6 Aplicaciones . . . 6

2 Wavelets 7
2.1 De la STFT a las wavelets . . . 7
2.2 Wavelets . . . 8
2.3 Transformada wavelet continua . . . 9
2.4 Transformada wavelet discreta . . . 10
2.5 La DWT como banco de filtros . . . 11
2.6 Aplicaciones . . . 13

3 Procesamiento de imágenes 15
3.1 La imagen como señal bidimensional . . . 15
3.2 Operaciones puntuales . . . 16
3.3 Histograma y ecualización . . . 16
3.4 Convolución bidimensional . . . 18
3.5 DFT bidimensional . . . 20
3.6 Filtrado en el dominio de la frecuencia . . . 21
3.7 Otras aplicaciones . . . 22

Bibliografía 23

1 Transformada de Fourier de tiempo corto

Los temas anteriores dan las herramientas básicas: la DFT para analizar el contenido en frecuencia (Tema 3) y los filtros para modificarlo (Tema 4). Este tema las aplica a señales reales, que tienen dos rasgos que hasta ahora hemos dejado de lado: su contenido en frecuencia *cambia con el tiempo*, y a veces no dependen del tiempo sino del *espacio*, como las imágenes.

1.1 Señales no estacionarias

La DFT de una señal completa dice qué frecuencias contiene, pero no cuándo aparece cada una. En la figura 1, x_1 tiene un tono de 50 Hz durante medio segundo seguido de uno de 120 Hz, y x_2 los mismos tonos en orden inverso. Los módulos de sus DFT son prácticamente iguales: la información temporal está escondida en la fase, de donde es imposible leerla.

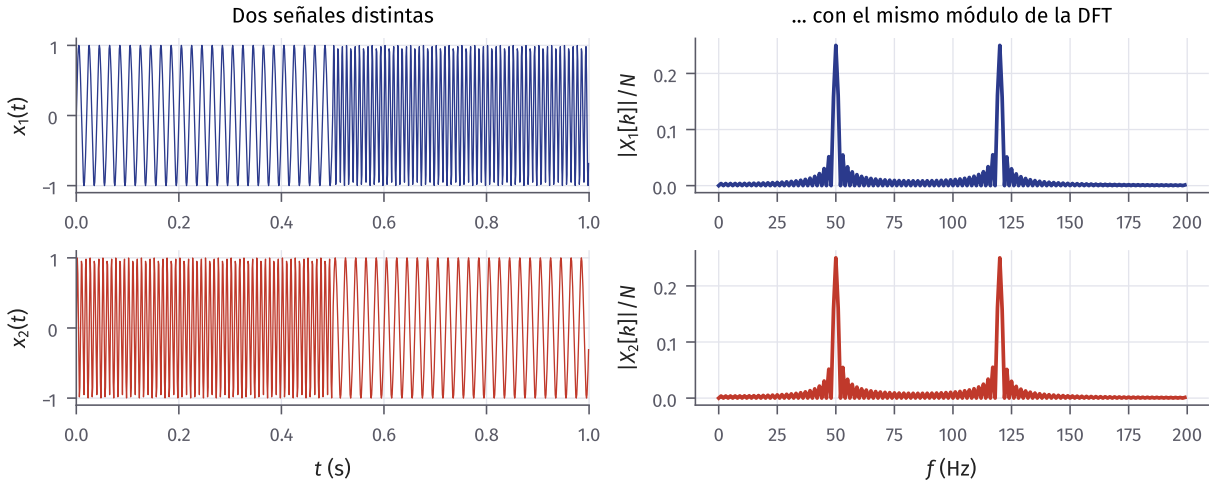


Figura 1. Dos señales distintas con el mismo módulo de la DFT.

La voz, la música, las vibraciones de una máquina que arranca o los ecos de un radar son *no estacionarias*: su espectro cambia con el tiempo (en el Tema 6 se da una definición precisa de estacionariedad). Para estudiarlas se analiza la señal *por tramos*: se multiplica por una ventana $w[n]$ de L muestras colocada en cada instante y se calcula la DFT del tramo (figura 2). Como vimos en el Tema 3, multiplicar por una ventana es convolucionar el espectro de la señal con el de la ventana, así que conviene usar ventanas suaves (Hann, Hamming) y no la rectangular, cuyos lóbulos laterales son altos.

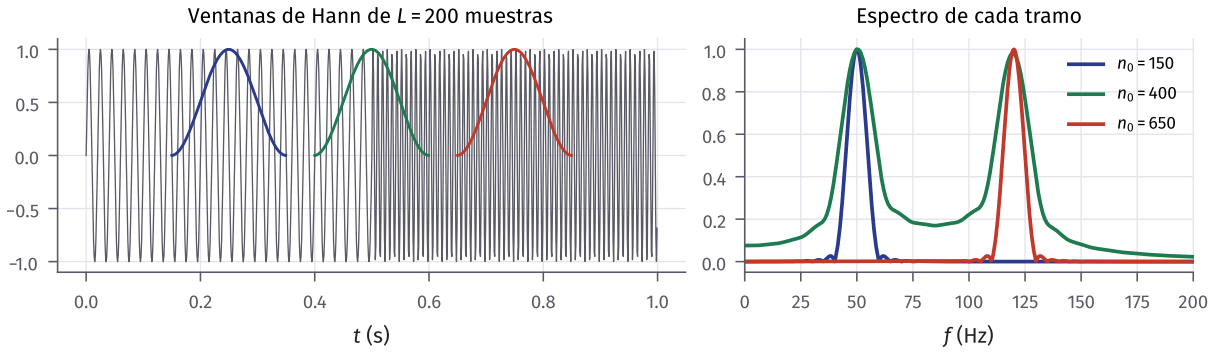


Figura 2. Tres ventanas de Hann sobre la señal x_1 y el espectro de cada tramo. El tramo central contiene el cambio de frecuencia y muestra los dos tonos.

Erratas de versiones anteriores

Las transparencias antiguas hablaban de «envenenamiento» de la señal donde quería decir *enventanado*, y llamaban `espectrogram` a la función `spectrogram` de Matlab.

1.2 Definición

Definición 1.1: Transformada de Fourier de tiempo corto (STFT)

En tiempo continuo, con una ventana $w(t)$ desplazada a τ,

$$X(\tau, \omega) = \int_{-\infty}^{\infty} x(t)\, w(t-\tau)\, \mathrm{e}^{-j\omega t}\, \mathrm{d}t.$$

En tiempo discreto, con una ventana de L muestras que avanza R muestras de un tramo al siguiente,

$$X(m, \Omega) = \sum_{n=-\infty}^{\infty} x[n]\, w[n - mR]\, \mathrm{e}^{-j\Omega n},$$

y, muestreando la frecuencia con $N \geq L$ puntos, $X[m,k] = X(m, 2\pi k/N)$, que se calcula con una DFT de N puntos de cada tramo.

Para cada tramo m hay un espectro, y para cada frecuencia k una secuencia en el tiempo: la STFT es una función de dos variables. Con frecuencia de muestreo f_s, el tramo m empieza en la muestra mR y está centrado en $t_m = (mR + L/2)/f_s$, y el índice k corresponde a $f_k = k f_s/N$. Algunos textos escriben la exponencial como $\mathrm{e}^{-j\Omega(n-mR)}$, es decir, miden el tiempo desde el principio de cada tramo; eso multiplica X por $\mathrm{e}^{j\Omega mR}$, que cambia la fase pero no el módulo.

Si la ventana se anula fuera de $0 \leq n < L$, la suma tiene solo L términos:

$$X[m,k] = \sum_{r=0}^{L-1} x[mR + r]\, w[r]\, \mathrm{e}^{-j2\pi k(mR+r)/N}.$$

Para una señal de N_x muestras, el número de tramos completos es

$$M = \left\lfloor \frac{N_x - L}{R} \right\rfloor + 1.$$

Una señal real de 2 s a $f_s = 1\,\mathrm{kHz}$ con $L = 128$, $R = 32$ y $N = 256$ da $M = \lfloor 1872/32 \rfloor + 1 = 59$ tramos, y de cada uno basta guardar las $N/2 + 1 = 129$ frecuencias entre 0 y $f_s/2$.

En código

STFT calculada tramo a tramo para un chirp que barre de 50 a 400 Hz en 2 s. La frecuencia del máximo de cada tramo sigue la frecuencia instantánea $50 + 175t$ Hz.

Python

```python
import numpy as np
from scipy import signal

fs = 1000
t = np.arange(0, 2, 1/fs)
x = signal.chirp(t, 50, 2, 400)  # 50->400
L, R, N = 128, 32, 256       # L, salto, FFT
w = np.hanning(L)
ini = np.arange(0, len(x) - L + 1, R)
X = np.array([np.fft.rfft(x[i:i+L]*w, N)
              for i in ini]).T   # (f, t)
f = np.fft.rfftfreq(N, 1/fs)
tm = (ini + L/2)/fs              # centros
pico = f[np.argmax(np.abs(X), axis=0)]
print(X.shape)                   # (129, 59)
print(np.c_[tm, pico][::10])     # f(t)
# Igual: signal.spectrogram(x, fs, w, ...)
```

Matlab

```matlab
fs = 1000;
t = 0:1/fs:2-1/fs;
x = chirp(t, 50, 2, 400);        % 50->400
L = 128; R = 32; N = 256;    % L, salto, FFT
w = hanning(L)';
ini = 1:R:numel(x)-L+1;
X = zeros(N/2+1, numel(ini));
for m = 1:numel(ini)
  S = fft(x(ini(m):ini(m)+L-1).*w, N);
  X(:, m) = S(1:N/2+1);          % (f, t)
end
f = (0:N/2)*fs/N;
tm = (ini - 1 + L/2)/fs;         % centros
[~, k] = max(abs(X));
disp(size(X))                    % 129 59
disp([tm(1:10:end); f(k(1:10:end))]')
% Igual: spectrogram(x, w, L-R, N, fs)
```

1.3 Espectrograma

El *espectrograma* es el cuadrado del módulo de la STFT, $|X[m,k]|^2$, representado sobre el plano tiempo-frecuencia con un mapa de colores, casi siempre en decibelios. En la figura 3 se distinguen tres componentes: el chirp aparece como una recta, el tono de 300 Hz como un segmento horizontal entre 0,6 y 1,2 s, y el clic de $t = 1{,}6$ s como una línea vertical, porque un impulso contiene todas las frecuencias. Las líneas verticales tenues en 0,6 y 1,2 s son los transitorios del encendido y apagado brusco del tono.

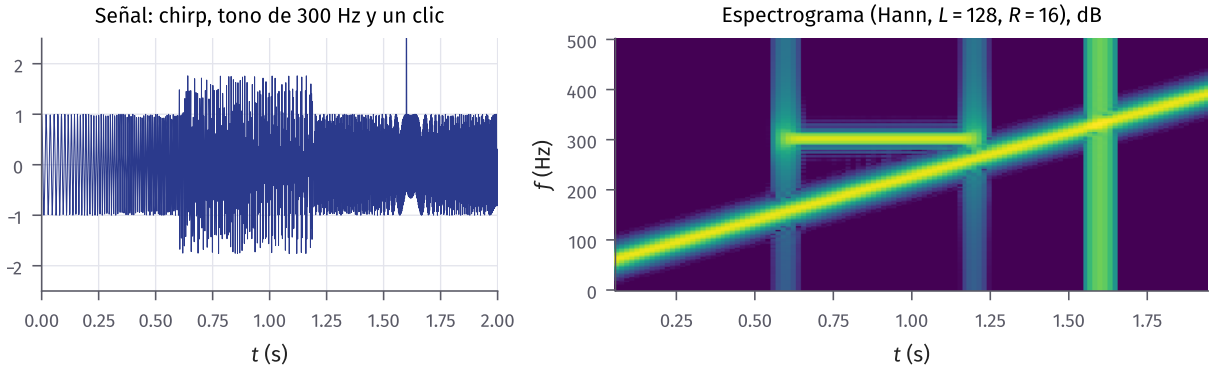


Figura 3. Señal de prueba y su espectrograma con una ventana de Hann de 128 muestras y salto de 16.

En Matlab el espectrograma lo calcula y dibuja `spectrogram(x, w, L-R, N, fs)`, cuyo tercer argumento es el solapamiento $L - R$; en Python, `scipy.signal.spectrogram` o la clase `ShortTimeFFT`. Octave no tiene `spectrogram`, sino `specgram`. Los ejemplos de estos apuntes calculan la STFT a mano para que se vea qué hacen.

1.4 Resolución en tiempo y en frecuencia

Los parámetros de la STFT son:

- La *longitud de la ventana* L, que fija la duración analizada, $\Delta t = L/f_s$, y la resolución en frecuencia. El lóbulo principal de una ventana de Hann mide $4f_s/L$ entre ceros, y dos tonos se distinguen si

están separados al menos la mitad, $\approx 2f_s/L$ (Tema 3).

- El *tipo de ventana*, que reparte el compromiso entre la anchura del lóbulo principal y la altura de los secundarios.
- El *salto* R o, equivalentemente, el *solapamiento* $L-R$. No cambia la resolución, sino lo densamente que se muestrea el eje de tiempos. Son habituales solapamientos del 50 al 75 %.
- El número de puntos de la DFT, $N \geq L$. Rellenar con ceros interpola el espectro, pero no separa mejor dos tonos.

La figura 4 muestra el compromiso. Con $L = 32$ (32 ms) el clic y los bordes del tono quedan bien localizados, pero el lóbulo mide 125 Hz y las frecuencias se emborronan. Con $L = 512$ el tono de 300 Hz es una raya fina, pero el chirp, que cambia 90 Hz durante la ventana, se ensancha, y el clic se extiende medio segundo.

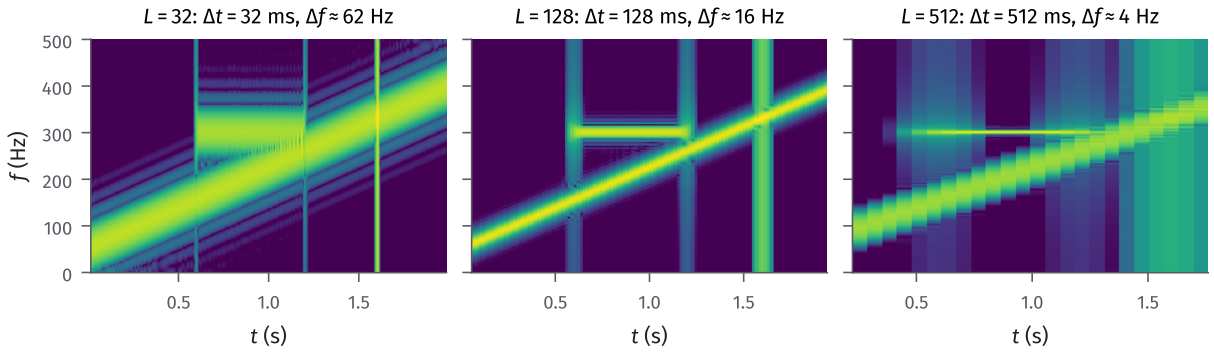


Figura 4. La misma señal con tres longitudes de ventana. Δf es la separación mínima entre dos tonos para la ventana de Hann.

Este compromiso es inevitable. Una señal de duración Δt tiene un espectro de anchura del orden de $1/\Delta t$: es el *principio de incertidumbre*. Si se miden las anchuras con las desviaciones típicas de $|w(t)|^2$ y de $|W(j\omega)|^2$ (normalizadas como densidades),

$$\sigma_t \, \sigma_\omega \geq \frac{1}{2},$$

con igualdad solo para la ventana gaussiana. La STFT usa la misma ventana en todas las frecuencias, de modo que su resolución es la misma en todo el plano tiempo-frecuencia. La longitud se elige según la señal: en voz se usan 20–30 ms, el tiempo durante el que el tracto vocal apenas cambia; en música, ventanas más largas para separar notas próximas.

Ejemplo 1.2: Una ventana imposible

Con $f_s = 1$ kHz y ventana de Hann, separar dos tonos a 20 Hz exige $2f_s/L \leq 20$, es decir, $L \geq 100$ muestras (100 ms). Separar dos clics a 10 ms exige una ventana más corta que su separación, $L \lesssim 10$. Ninguna longitud cumple ambas condiciones: hay que calcular dos espectrogramas o usar un análisis con resolución variable, como el de las wavelets (sección 2).

1.5 Síntesis por solapamiento y suma

La STFT se puede invertir. La IDFT del tramo m devuelve $x[n]\, w[n - mR]$; si se suman todos los tramos,

$$\sum_m x[n]\, w[n - mR] = x[n] \sum_m w[n - mR],$$

que vale $C\,x[n]$ si las ventanas desplazadas suman una constante:

$$\sum_m w[n - mR] = C \qquad \text{para todo } n.$$

Es la condición *COLA* (*constant overlap-add*), y el método se llama *solapamiento y suma*. La cumplen la ventana de Hann periódica $w[n] = \frac{1}{2} - \frac{1}{2}\cos(2\pi n/L)$, $0 \le n < L$, con $R = L/2$ (suma 1) o $R = L/4$ (suma 2), la de Hamming periódica con $R = L/2$ (suma 1,08) y la rectangular con $R = L$ (figura 5). Gracias a ella se puede modificar la señal en el plano tiempo-frecuencia (atenuar el ruido en los puntos donde domina, eliminar un tono durante un intervalo) y volver al tiempo. Es la base de muchos reductores de ruido y del *vocoder* de fase, que cambia el tempo de una grabación sin cambiar su tono.

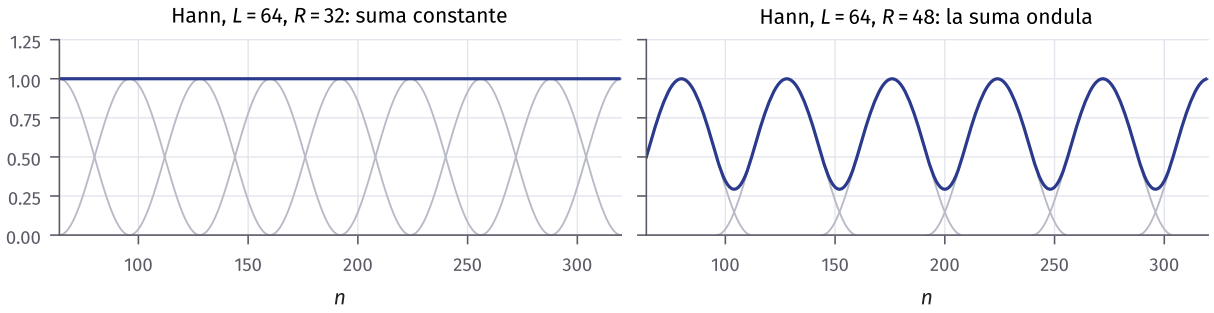


Figura 5. Ventanas de Hann desplazadas (gris) y su suma (azul). Con $R = L/2$ la suma es constante; con $R = 3L/4$, no.

Ventana periódica y simétrica

La ventana de Hann *simétrica* de longitud L, la de `hann(L)` en Matlab, tiene ceros en $n = 0$ y $n = L - 1$ y no cumple COLA exactamente. La *periódica*, `hann(L,'periodic')`, es la simétrica de longitud $L + 1$ sin la última muestra. Para analizar da igual; para resintetizar conviene la periódica.

1.6 Aplicaciones

Marcación por tonos. En la marcación DTMF cada tecla del teléfono suma una sinusoide de fila (697, 770, 852 o 941 Hz) y una de columna (1209, 1336 o 1477 Hz). El espectrograma de la figura 6 muestra en qué instante se pulsó cada tecla y cuál: la «2» es 697 + 1336 Hz, el «0» 941 + 1336 y el «5» 770 + 1336. Para separar 697 y 770 Hz, que distan 73 Hz, una ventana de Hann necesita $L \ge 2f_s/73 \approx 220$ muestras a 8 kHz (27 ms), menos que la duración mínima de un tono.

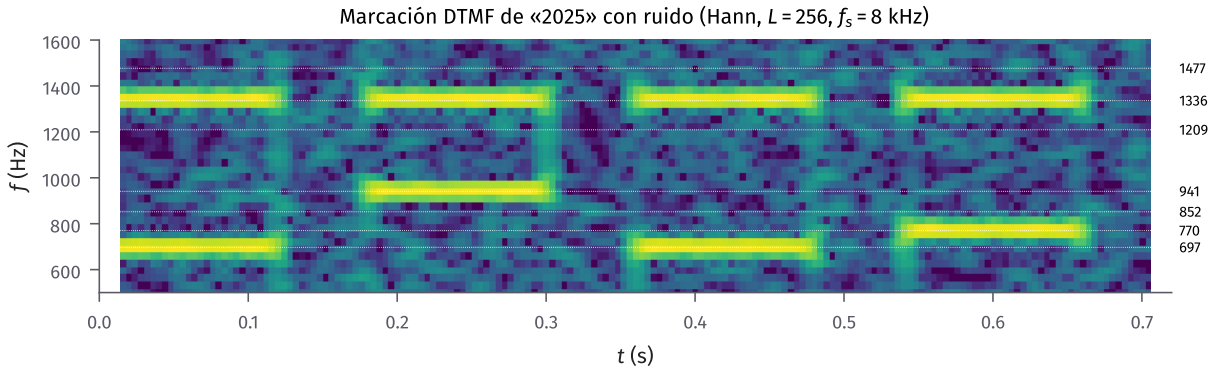


Figura 6. Espectrograma de la marcación de «2025» con ruido.

En código

Reconocimiento de una tecla a partir de 100 ms de señal con ruido: se busca el máximo del espectro en la banda de las filas y en la de las columnas. Los receptores reales usan el algoritmo de Goertzel, que calcula solo las 7 componentes de la DFT que interesan.

Python

```python
import numpy as np

fs = 8000
t = np.arange(0, 0.1, 1/fs)   # 800 muestras
x = np.sin(2*np.pi*852*t) \
    + np.sin(2*np.pi*1336*t)     # tecla 8
rng = np.random.default_rng(0)
x += 0.5*rng.standard_normal(t.size)
N = 4096
w = np.hamming(x.size)
X = np.abs(np.fft.rfft(x*w, N))
f = np.fft.rfftfreq(N, 1/fs)
baja = (f > 650) & (f < 1000)
alta = (f > 1150) & (f < 1550)
fb = f[baja][np.argmax(X[baja])]
fa = f[alta][np.argmax(X[alta])]
filas = np.array([697, 770, 852, 941])
cols = np.array([1209, 1336, 1477])
teclas = ["123", "456", "789", "*0#"]
i = np.argmin(abs(filas - fb))
j = np.argmin(abs(cols - fa))
print(fb, fa, teclas[i][j])      # ... 8
```

Matlab

```matlab
fs = 8000;
t = 0:1/fs:0.1-1/fs;          % 800 muestras
x = sin(2*pi*852*t) ...
    + sin(2*pi*1336*t);          % tecla 8
x = x + 0.5*randn(size(t));
N = 4096;
w = hamming(numel(x))';
X = abs(fft(x.*w, N));
X = X(1:N/2+1);
f = (0:N/2)*fs/N;
baja = find(f > 650 & f < 1000);
alta = find(f > 1150 & f < 1550);
[~, ib] = max(X(baja)); fb = f(baja(ib));
[~, ia] = max(X(alta)); fa = f(alta(ia));
filas = [697 770 852 941];
cols = [1209 1336 1477];
teclas = ['123'; '456'; '789'; '*0#'];
[~, i] = min(abs(filas - fb));
[~, j] = min(abs(cols - fa));
disp([fb fa]); disp(teclas(i, j))  % 8
```

Otras aplicaciones. En *voz*, el espectrograma muestra los formantes (resonancias del tracto vocal) y el tono de la voz, y los reconocedores automáticos parten de una STFT. En *música*, permite identificar notas e instrumentos. En *mantenimiento predictivo*, el arranque de un motor es un chirp y un rodamiento dañado produce armónicos que crecen con el tiempo. En *radar y sonar*, el desplazamiento Doppler a lo largo del tiempo revela la velocidad de un blanco.

Ejercicios 1–5.

2 Wavelets

2.1 De la STFT a las wavelets

La STFT fija la ventana antes de analizar y la usa en todas las frecuencias. Pero muchas señales mezclan componentes lentas y largas (una tendencia, un tono grave) con detalles rápidos y breves (un clic, un salto, un borde). Lo natural sería analizar las bajas frecuencias con ventanas largas y las altas con ventanas cortas, de forma que cada análisis abarque aproximadamente el mismo número de periodos. Además, la sinusoide no siempre es la mejor forma con la que comparar: un latido del corazón o un escalón se parecen más a otras ondas.

La figura 7 resume la diferencia. Cada rectángulo representa la resolución de un coeficiente: la STFT reparte el plano en rectángulos iguales; las wavelets usan rectángulos estrechos y altos en alta fre-

cuencia y anchos y bajos en baja frecuencia. El área es la misma en todos, como exige el principio de incertidumbre.

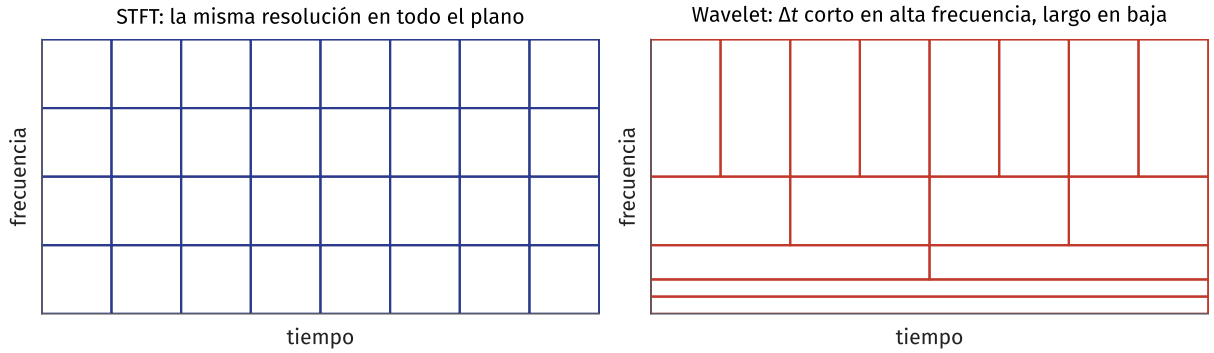


Figura 7. Reparto del plano tiempo-frecuencia de la STFT y de la transformada wavelet.

2.2 Wavelets

Definición 2.1: Wavelet

Una *wavelet madre* $\psi(t)$ es una función de energía finita, de duración limitada o que decae rápidamente, y de media nula. A partir de ella se forma la familia

$$\psi_{a,b}(t) = \frac{1}{\sqrt{a}}\,\psi\!\left(\frac{t-b}{a}\right), \qquad a>0,\ b\in\mathbb{R},$$

donde a es la *escala* y b el *desplazamiento*. El factor $1/\sqrt{a}$ hace que todas tengan la misma energía.

La palabra significa «ondita»: oscila, como una sinusoide, pero solo durante un intervalo. Con $a<1$ la wavelet se comprime y oscila más deprisa; con $a>1$ se dilata (figura 9). Si ψ oscila con frecuencia ω_0, $\psi_{a,b}$ oscila con ω_0/a: la escala es inversamente proporcional a la frecuencia.

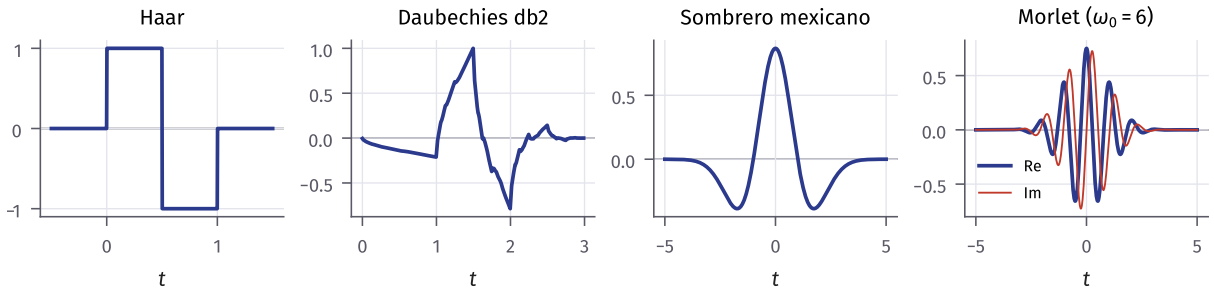


Figura 8. Algunas wavelets madre. La de Morlet es compleja.

Las wavelets más usadas (figura 8) son:

- *Haar*: $\psi(t)=1$ en $[0,\frac{1}{2})$, -1 en $[\frac{1}{2},1)$ y 0 en el resto. La más sencilla; discontinua.
- *Daubechies* db$P$: soporte de longitud $2P-1$ y P momentos nulos (véase más abajo); db1 es Haar. No tienen expresión cerrada: se definen por sus filtros.
- *Sombrero mexicano*: la segunda derivada de una gaussiana cambiada de signo, $\psi(t)\propto(1-t^2)\mathrm{e}^{-t^2/2}$.

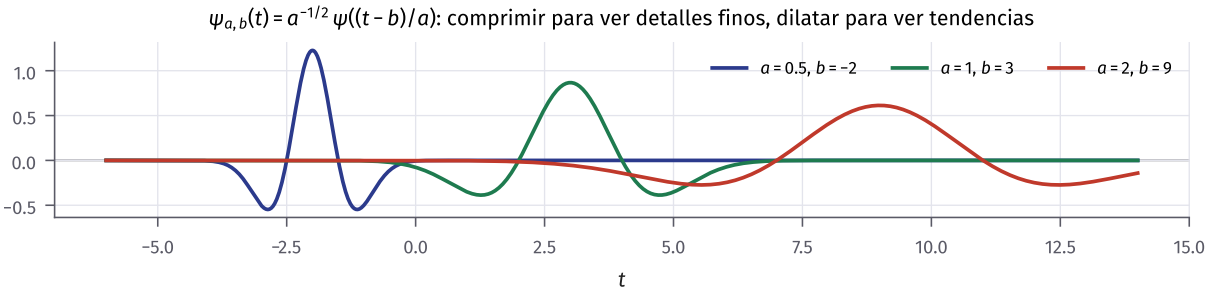


Figura 9. Sombrero mexicano con tres escalas y desplazamientos.

- *Morlet*: una exponencial compleja con envolvente gaussiana, $\psi(t) = \pi^{-1/4} e^{j\omega_0 t} e^{-t^2/2}$, normalmente con $\omega_0 = 6$. Es la más parecida a la STFT con ventana gaussiana.

2.3 Transformada wavelet continua

Definición 2.2: Transformada wavelet continua

$$W_x(a,b) = \int_{-\infty}^{\infty} x(t)\,\psi_{a,b}^*(t)\,\mathrm{d}t = \frac{1}{\sqrt{a}} \int_{-\infty}^{\infty} x(t)\,\psi^*\!\left(\frac{t-b}{a}\right)\mathrm{d}t.$$

$W_x(a,b)$ es la correlación de x con la wavelet escalada, evaluada en b: mide cuánto se parece la señal, alrededor de $t = b$, a una onda de la forma de ψ y anchura proporcional a a. La STFT correla con $w(t-\tau)e^{j\omega t}$, que depende de tres parámetros (la posición, la frecuencia y la longitud de la ventana, fijada de antemano); la transformada wavelet, de dos.

El *escalograma* es $|W_x(a,b)|^2$ sobre el plano (b,a). Se suele dibujar con la frecuencia equivalente en un eje logarítmico: para la Morlet, con a en segundos, $f \approx \omega_0/(2\pi a)$. En la figura 10 se compara con el espectrograma: el clic se estrecha al subir la frecuencia y el tono de 20 Hz se resuelve en frecuencia mejor que el de 120 Hz. Para la Morlet, la envolvente $e^{-t^2/(2a^2)}$ tiene desviación típica $\sigma_t = a$ y su espectro, $\sigma_\omega = 1/a$: el producto es constante y el área de cada «rectángulo» de la figura 7 también.

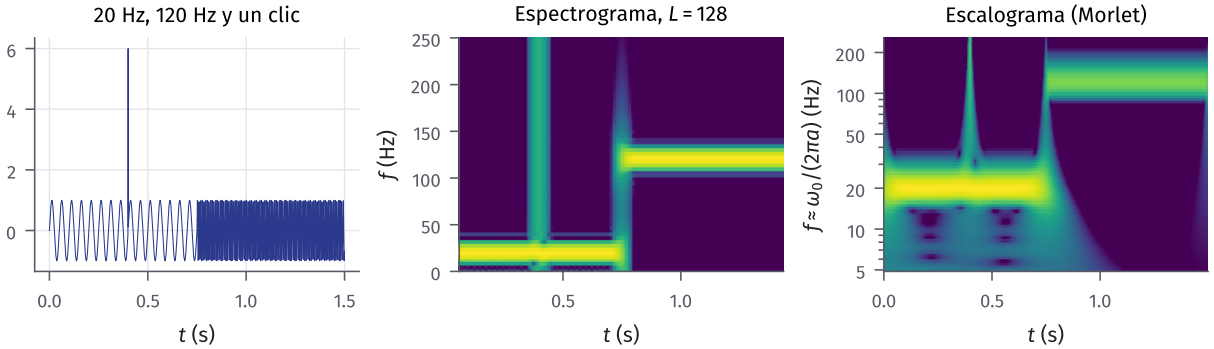


Figura 10. Señal con un tono de 20 Hz, uno de 120 Hz y un clic: espectrograma y escalograma con la wavelet de Morlet.

Para que la transformada se pueda invertir, ψ debe cumplir la *condición de admisibilidad*

$$C_\psi = \int_{-\infty}^{\infty} \frac{|\Psi(j\omega)|^2}{|\omega|}\,\mathrm{d}\omega < \infty,$$

donde $\Psi(j\omega)$ es la CTFT de ψ. Si Ψ es continua, la integral solo converge si $\Psi(j0) = 0$, es decir, si

$\int \psi(t)\,\mathrm{d}t = 0$: la wavelet tiene media nula y por tanto oscila. Entonces

$$x(t) = \frac{1}{C_\psi}\int_0^\infty\int_{-\infty}^\infty W_x(a,b)\,\psi_{a,b}(t)\,\frac{\mathrm{d}b\;\mathrm{d}a}{a^2}.$$

Definición 2.3: Momentos nulos

Una wavelet tiene P momentos nulos si

$$\int_{-\infty}^\infty t^p\,\psi(t)\,\mathrm{d}t = 0, \qquad p = 0, 1, \ldots, P-1.$$

Si ψ tiene P momentos nulos, la transformada de cualquier polinomio de grado menor que P es cero. Una señal suave se parece localmente a un polinomio, así que sus coeficientes son casi nulos en las escalas finas, y solo destacan las discontinuidades y los cambios bruscos. Haar tiene un momento nulo ($\int t\,\psi(t)\,\mathrm{d}t = \frac{1}{8} - \frac{3}{8} = -\frac{1}{4} \neq 0$), el sombrero mexicano dos y db$P$, P.

Erratas de versiones anteriores

La admisibilidad aparecía con ω en el denominador, sin valor absoluto, y explicada como que $|\Psi|^2$ «decae más rápidamente que $1/\omega$»; lo que exige en realidad es que Ψ se anule en $\omega = 0$. Los momentos nulos se definían con $p = 0, \ldots, N$ para «N momentos», que son $N+1$ condiciones. Y se decía que la wavelet es «irregular y asimétrica», que no es una condición: el sombrero mexicano y la Morlet son simétricas.

2.4 Transformada wavelet discreta

La CWT es muy redundante: una función de una variable se convierte en una de dos. Basta muestrear escalas y desplazamientos en una rejilla *diádica*, $a = 2^j$ y $b = k\,2^j$ con $j, k \in \mathbb{Z}$:

$$\psi_{j,k}(t) = 2^{-j/2}\,\psi\left(2^{-j}t - k\right).$$

Para wavelets bien elegidas (Haar, Daubechies y otras) estas funciones forman una *base ortonormal*:

$$\int_{-\infty}^\infty \psi_{j,k}(t)\,\psi_{j',k'}(t)\,\mathrm{d}t = \delta[j-j']\,\delta[k-k'].$$

Cualquier señal de energía finita se escribe entonces como

$$x(t) = \sum_j\sum_k d_j[k]\,\psi_{j,k}(t), \qquad d_j[k] = \int_{-\infty}^\infty x(t)\,\psi_{j,k}(t)\,\mathrm{d}t,$$

y, por ser la base ortonormal, se cumple la igualdad de Parseval: $\int |x(t)|^2\,\mathrm{d}t = \sum_{j,k} |d_j[k]|^2$.

Erratas de versiones anteriores

Las transparencias antiguas definían $\psi_{j,k}(t) = \psi(2^{-j}t - k)$, sin el factor $2^{-j/2}$, con lo que la base no es ortonormal, y escribían la ortonormalidad sin el $\mathrm{d}t$ y con deltas de Dirac en lugar de deltas de Kronecker. En la descomposición por niveles llamaban a la señal unas veces $x(t)$ y otras $f(t)$.

Análisis multirresolución. Además de la wavelet se define una *función de escala* $\varphi(t)$ (a veces llamada «wavelet padre»), de media 1, cuyas versiones $\varphi_{j,k}(t) = 2^{-j/2}\varphi(2^{-j}t - k)$ recogen lo que queda de la señal por encima de una escala. Para Haar, φ es el pulso rectangular de $[0, 1)$. Se definen:

- el *detalle* del nivel j, $D_j(t) = \sum_k d_j[k]\,\psi_{j,k}(t)$, lo que la señal tiene a la escala 2^j;
- la *aproximación* del nivel J, $A_J(t) = \sum_k a_J[k]\,\varphi_{J,k}(t)$, con $a_J[k] = \int x(t)\varphi_{J,k}(t)\,\mathrm{d}t$: la señal sin los detalles más finos que 2^J.

Partiendo de una señal muestreada, que se identifica con la aproximación de nivel 0,

$$x = A_J + D_J + D_{J-1} + \cdots + D_1, \qquad A_{j-1} = A_j + D_j.$$

La figura 11 muestra una descomposición de Haar en cuatro niveles: la aproximación A_4 conserva la forma general; los saltos aparecen en todos los detalles; la oscilación rápida del final, en D_2 y D_3.

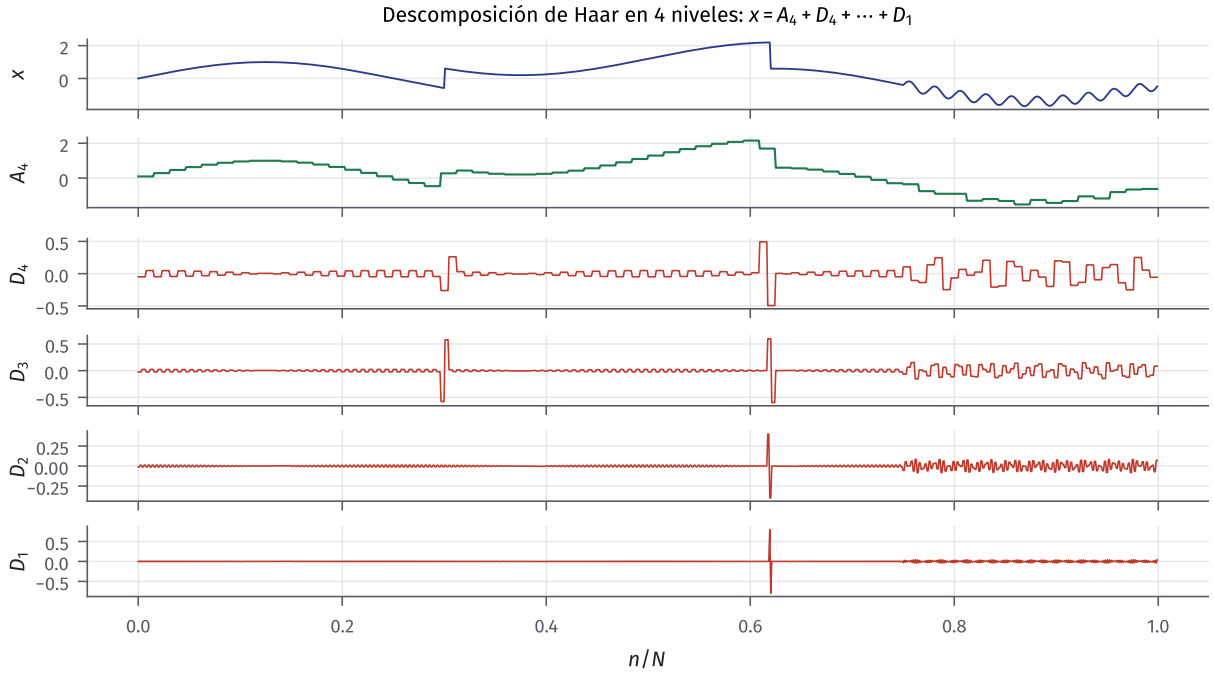


Figura 11. Señal, aproximación y detalles de una descomposición de Haar en cuatro niveles. Cada componente tiene la longitud de la señal.

2.5 La DWT como banco de filtros

φ y ψ cumplen las *ecuaciones de dos escalas*

$$\varphi(t) = \sqrt{2}\sum_n h_0[n]\,\varphi(2t - n), \qquad \psi(t) = \sqrt{2}\sum_n h_1[n]\,\varphi(2t - n),$$

donde h_0 es un filtro paso bajo y $h_1[n] = (-1)^n h_0[L-1-n]$ un paso alto de la misma longitud L. De ellas se deduce el *algoritmo de Mallat*, que calcula los coeficientes de un nivel a partir de la aproximación del anterior sin calcular ninguna integral:

$$a_j[k] = \sum_n h_0[n - 2k]\,a_{j-1}[n], \qquad d_j[k] = \sum_n h_1[n - 2k]\,a_{j-1}[n].$$

Es una convolución con $h_0[-n]$ y $h_1[-n]$ seguida de un *diezmado por 2* (quedarse con una de cada dos muestras). La inversa intercala ceros (interpola por 2), filtra con h_0 y h_1 y suma.

Propiedad 2.4: DWT de Haar

Con $h_0 = \frac{1}{\sqrt{2}}(1,1)$ y $h_1 = \frac{1}{\sqrt{2}}(1,-1)$, un nivel de la DWT de una secuencia de longitud par es

$$a_1[k] = \frac{x[2k] + x[2k+1]}{\sqrt{2}}, \qquad d_1[k] = \frac{x[2k] - x[2k+1]}{\sqrt{2}},$$

y la inversa,

$$x[2k] = \frac{a_1[k] + d_1[k]}{\sqrt{2}}, \qquad x[2k+1] = \frac{a_1[k] - d_1[k]}{\sqrt{2}}.$$

La aproximación es la media de cada pareja (por $\sqrt{2}$) y el detalle, su semidiferencia.

Ejemplo 2.5: Tres niveles de Haar

Con $x = (4, 6, 10, 12, 8, 6, 5, 5)$:

- Nivel 1: $a_1 = \frac{1}{\sqrt{2}}(10, 22, 14, 10)$, $d_1 = \frac{1}{\sqrt{2}}(-2, -2, 2, 0)$.
- Nivel 2, sobre a_1: $a_2 = \frac{1}{2}(10 + 22,\ 14 + 10) = (16, 12)$, $d_2 = \frac{1}{2}(10 - 22,\ 14 - 10) = (-6, 2)$.
- Nivel 3: $a_3 = \frac{28}{\sqrt{2}} = 19{,}80$ y $d_3 = \frac{4}{\sqrt{2}} = 2{,}83$.

La energía se conserva: $\sum x^2 = 446$ y $392 + 8 + (36 + 4) + \frac{4+4+4+0}{2} = 446$. Casi toda está en a_3, la media.

Cada nivel parte la banda en dos (figura 12): d_1 recoge aproximadamente las frecuencias de $[\pi/2, \pi]$, d_2 las de $[\pi/4, \pi/2]$, y así hasta a_J, que recoge $[0, \pi/2^J]$. Los filtros de Haar separan mal las bandas; los de Daubechies más largos, mejor. En todos los casos $|H_0|^2 + |H_1|^2 = 2$: la energía que uno quita la recoge el otro.

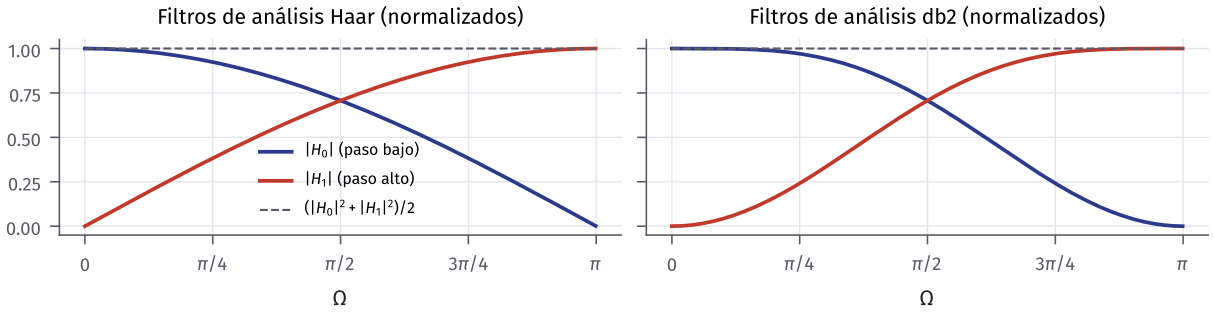


Figura 12. Respuesta de los filtros de análisis de Haar y de db2.

El número total de coeficientes es $\frac{N}{2} + \frac{N}{4} + \cdots + \frac{N}{2^J} + \frac{N}{2^J} = N$: la DWT no añade redundancia, y el coste es $O(N)$, menor aún que el de la FFT. Los coeficientes del filtro db2 son

$$h_0 = \frac{1}{4\sqrt{2}}\left(1 + \sqrt{3},\ 3 + \sqrt{3},\ 3 - \sqrt{3},\ 1 - \sqrt{3}\right),$$

y su paso alto h_1 cumple $\sum_n h_1[n] = 0$ y $\sum_n n\, h_1[n] = 0$: dos momentos nulos, de modo que los detalles de una rampa son nulos, salvo en los extremos. Con Haar, $x[n] = n$ da $d_1[k] = -1/\sqrt{2}$ en todos los k.

En código

DWT de Haar en cuatro niveles, comprobación de Parseval y de la reconstrucción. En Matlab, la Wavelet Toolbox ofrece `dwt`, `wavedec`, `waverec`, `wrcoef` y `cwt`; en Python, el paquete PyWavelets (`pywt.wavedec`, `pywt.cwt`).

Python

```python
import numpy as np
def dwt_haar(x, J):       # [d1, ..., dJ, aJ]
    a, c = np.asarray(x, float), []
    for _ in range(J):            # análisis
        c.append((a[0::2] - a[1::2])/2**.5)
        a = (a[0::2] + a[1::2])/2**.5
    return c + [a]

def idwt_haar(c):
    a = c[-1]
    for d in reversed(c[:-1]):    # síntesis
        x = np.empty(2*a.size)
        x[0::2] = (a + d)/2**.5
        x[1::2] = (a - d)/2**.5
        a = x
    return a
n = np.arange(1024)
x = np.sin(2*np.pi*n/512) + (n > 300)
c = dwt_haar(x, 4)
print([v.size for v in c])      # 512 ... 64
E = [np.sum(v**2) for v in c]
print(np.round(E, 2), np.sum(x**2))
print(abs(idwt_haar(c) - x).max())  # ~1e-15
```

Matlab

```matlab
n = 0:1023;
x = sin(2*pi*n/512) + (n > 300);
J = 4; c = cell(1, J+1); a = x;
for j = 1:J                       % análisis
  c{j} = (a(1:2:end)-a(2:2:end))/sqrt(2);
  a = (a(1:2:end)+a(2:2:end))/sqrt(2);
end
c{J+1} = a;
disp(cellfun(@numel, c))        % 512 ... 64
E = cellfun(@(v) sum(v.^2), c);
disp([E, sum(x.^2)])             % Parseval
for j = J:-1:1                    % síntesis
  y = zeros(1, 2*numel(a));
  y(1:2:end) = (a + c{j})/sqrt(2);
  y(2:2:end) = (a - c{j})/sqrt(2);
  a = y;
end
disp(max(abs(a - x)))             % ~1e-15
```

2.6 Aplicaciones

Eliminación de ruido. La DWT concentra la energía de una señal con pocas discontinuidades en unos pocos coeficientes grandes, mientras que el ruido blanco, al ser la base ortonormal, reparte la suya por igual entre todos. Anulando los coeficientes de detalle pequeños y reconstruyendo se elimina gran parte del ruido. Hay dos reglas:

$$\text{umbral duro: } \hat{d} = \begin{cases} d, & |d| > u \\ 0, & |d| \le u \end{cases} \qquad \text{umbral suave: } \hat{d} = \operatorname{sgn}(d)\max(|d| - u, 0).$$

Un umbral habitual es el universal de Donoho y Johnstone, $u = \hat{\sigma}\sqrt{2\ln N}$, con la desviación típica del ruido estimada a partir de los detalles más finos, que son casi todo ruido: $\hat{\sigma} = \text{mediana}|d_1[k]|/0{,}6745$. A diferencia de un paso bajo, que quita el ruido y también suaviza los saltos, el umbral conserva los saltos (figura 13).



Figura 13. Señal escalonada con ruido, filtrada con una media móvil y con umbral sobre la DWT de Haar. En rojo, la señal sin ruido.

En código

Umbral duro sobre cinco niveles de Haar. El error cuadrático medio baja de 0,40 a 0,12.

Python

```python
import numpy as np

N = 1024
n = np.arange(N)
x = 1.5*(n > 100) - 2.2*(n > 240) \
    + 1.6*(n > 410) + 1.2*(n > 512)
rng = np.random.default_rng(4)
y = x + 0.4*rng.standard_normal(N)
c, a = [], y                          # análisis
for j in range(5):
    c.append((a[0::2] - a[1::2])/2**.5)
    a = (a[0::2] + a[1::2])/2**.5
s = np.median(abs(c[0]))/0.6745   # sigma
u = s*np.sqrt(2*np.log(N))        # umbral
for d in reversed(c):             # síntesis
    d = np.where(abs(d) > u, d, 0)  # duro
    z = np.empty(2*a.size)
    z[0::2] = (a + d)/2**.5
    z[1::2] = (a - d)/2**.5
    a = z
rms = lambda e: np.sqrt(np.mean(e**2))
print(s, u)                       # 0.40 1.49
print(rms(y - x), rms(a - x))     # 0.40 0.12
```

Matlab

```matlab
N = 1024; n = 0:N-1;
x = 1.5*(n > 100) - 2.2*(n > 240) ...
    + 1.6*(n > 410) + 1.2*(n > 512);
y = x + 0.4*randn(1, N);
J = 5; c = cell(1, J+1); a = y;
for j = 1:J
  c{j} = (a(1:2:end)-a(2:2:end))/sqrt(2);
  a = (a(1:2:end)+a(2:2:end))/sqrt(2);
end
s = median(abs(c{1}))/0.6745;     % sigma
u = s*sqrt(2*log(N));             % umbral
for j = J:-1:1                    % duro
  d = c{j}.*(abs(c{j}) > u);
  z = zeros(1, 2*numel(a));
  z(1:2:end) = (a + d)/sqrt(2);
  z(2:2:end) = (a - d)/sqrt(2);
  a = z;
end
rms = @(e) sqrt(mean(e.^2));
disp([s u])                       % 0.40 1.49
disp([rms(y-x) rms(a-x)])         % 0.40 0.12
```

Compresión. Si se guardan solo los coeficientes mayores, la señal se reconstruye con poco error (figura 14). JPEG 2000 aplica una DWT bidimensional con wavelets biortogonales (la CDF 9/7) y codifica los coeficientes; a igual tamaño da menos artefactos que la DCT por bloques de JPEG.

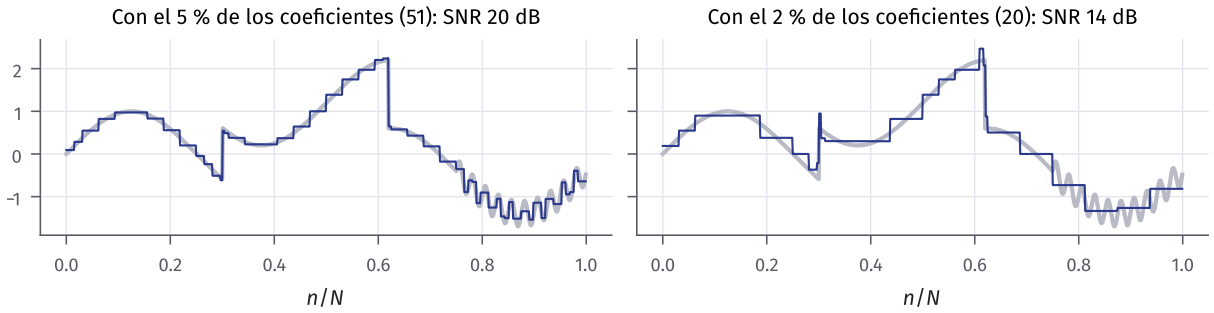


Figura 14. Reconstrucción a partir del 5 y del 2 % de los coeficientes de Haar.

Detección. Los saltos y picos producen coeficientes grandes en todas las escalas, y las wavelets se usan para detectar los complejos QRS de un electrocardiograma o los fallos incipientes en una máquina. La elección de la wavelet depende de la forma de lo que se busca y del número de momentos nulos necesarios.

Ejercicios 6–10.

3 Procesamiento de imágenes

3.1 La imagen como señal bidimensional

Una imagen es una señal que depende de dos variables espaciales. Una cámara la muestrea con una rejilla de sensores y cuantifica cada muestra: el resultado es una matriz $f[m, n]$ de $M \times N$ *píxeles*, con $m = 0, \dots, M-1$ la fila, contada de arriba abajo, y $n = 0, \dots, N-1$ la columna (figura 15). En Matlab los índices empiezan en 1: el píxel $f[m, n]$ es `f(m+1, n+1)`.

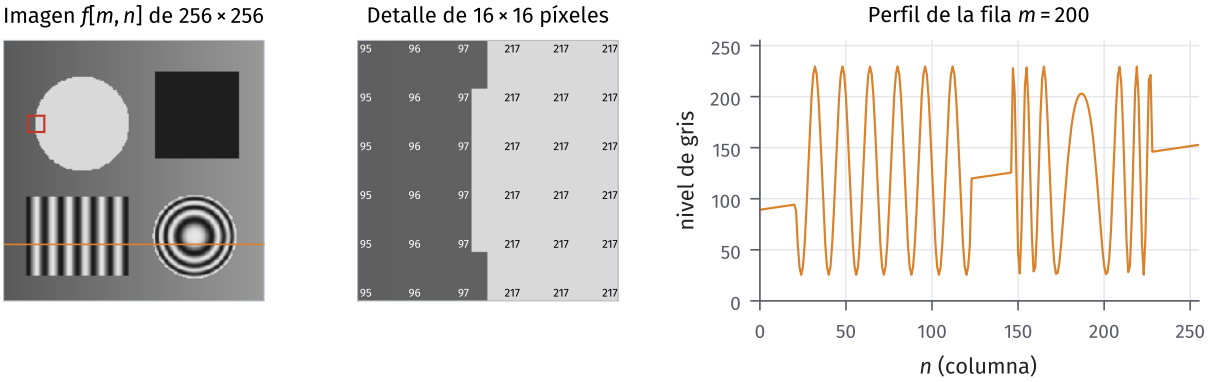


Figura 15. La imagen de prueba de los ejemplos, un detalle de 16 × 16 píxeles con algunos de sus valores (de 0 a 255) y el perfil de una fila.

- *Escala de grises*: un valor por píxel, entero de 8 bits (`uint8`, de 0, negro, a 255, blanco), de 16 bits, o real en $[0, 1]$.
- *Binaria*: 0 o 1.
- *Color*: tres canales, normalmente rojo, verde y azul (RGB), es decir, una matriz de $M \times N \times 3$. Hay otros espacios de color, como Lab (luminosidad y dos componentes de color) o YCbCr (el de JPEG).

El *muestreo espacial* tiene las mismas reglas que el temporal (Tema 2): la frecuencia espacial se mide en ciclos por píxel, o por milímetro, y un detalle con periodo menor que dos píxeles produce *aliasing*, el efecto moiré que aparece al fotografiar una tela de rayas finas. La *cuantificación* con b bits da 2^b niveles; con pocos niveles aparecen contornos falsos en los degradados.

Sin comprimir, una imagen ocupa $M \cdot N \cdot c \cdot b/8$ bytes, con c canales: una imagen en color de 1920×1080 a 8 bits ocupa 6,2 MB. Por eso se guardan comprimidas: sin pérdidas (PNG, TIFF), eliminando redundancia, o con pérdidas (JPEG, JPEG 2000), descartando lo que menos se percibe. Matlab lee y escribe imágenes con `imread` e `imwrite` y las muestra con `imshow` o `image`; Python, con `matplotlib.pyplot.imread` e `imshow` o con el paquete Pillow.

En código

Los ejemplos usan una imagen sintética de 256 × 256 con un degradado, un círculo, un rectángulo, unas franjas verticales y un patrón de anillos, para no depender de ficheros externos.

Python

```python
import numpy as np


def imagen_prueba(N=256):
    """Imagen sintética N x N en [0, 1]."""
    n = np.arange(N)/N
    x, y = np.meshgrid(n, n)
    f = 0.35 + 0.25*x            # degradado
    f[(x-.3)**2 + (y-.32)**2 < .18**2] = .85
    f[(x > 0.58) & (x < 0.9)
      & (y > 0.12) & (y < 0.45)] = 0.12
    fr = (y > 0.6) & (y < 0.9) \
        & (x > 0.08) & (x < 0.48)
    f[fr] = .5 + .4*np.cos(2*np.pi*16*x[fr])
    r = np.hypot(x - 0.73, y - 0.75)
    an = r < 0.16
    f[an] = 0.5 + 0.4*np.cos(2*np.pi*120
                             * r[an]**2)
    return f


if __name__ == "__main__":
    f = imagen_prueba()
    print(f.shape, f.min(), f.max())
```

Matlab

```matlab
function f = imagen_prueba(N)
% Imagen sintética de N x N en [0, 1].
if nargin < 1, N = 256; end
[x, y] = meshgrid((0:N-1)/N);
f = 0.35 + 0.25*x;               % degradado
f((x-0.3).^2 + (y-0.32).^2 < 0.18^2) = .85;
f(x > 0.58 & x < 0.9 ...
  & y > 0.12 & y < 0.45) = 0.12;
fr = y > 0.6 & y < 0.9 ...
     & x > 0.08 & x < 0.48;
f(fr) = 0.5 + 0.4*cos(2*pi*16*x(fr));
r = hypot(x - 0.73, y - 0.75);
an = r < 0.16;
f(an) = 0.5 + 0.4*cos(2*pi*120*r(an).^2);
end
```

Erratas de versiones anteriores

Las transparencias antiguas decían «Introducciónn», «repesentación» y «perdida» por «pérdida», y ponían «(ej: *tiff*)» como ejemplo de los formatos con pérdidas, además de los sin pérdidas.

3.2 Operaciones puntuales

Una *operación puntual* transforma cada píxel con una función de su propio valor, $s = T(r)$, sin mirar a los vecinos (figura 16). Con valores en $[0, 1]$:

- *negativo*, $s = 1 - r$;
- *corrección gamma*, $s = r^\gamma$: con $\gamma < 1$ aclara los tonos oscuros, con $\gamma > 1$ oscurece los claros;
- *estiramiento lineal* del intervalo $[r_1, r_2]$ a $[0, 1]$, recortando fuera de él (es lo que hace `imadjust` de Matlab);
- *umbralización*, $s = 1$ si $r > u$ y 0 si no, que da una imagen binaria.

3.3 Histograma y ecualización

Definición 3.1: Histograma

Si una imagen de $M \times N$ píxeles tiene L niveles $r = 0, \ldots, L-1$, su *histograma* $h[r]$ es el número de

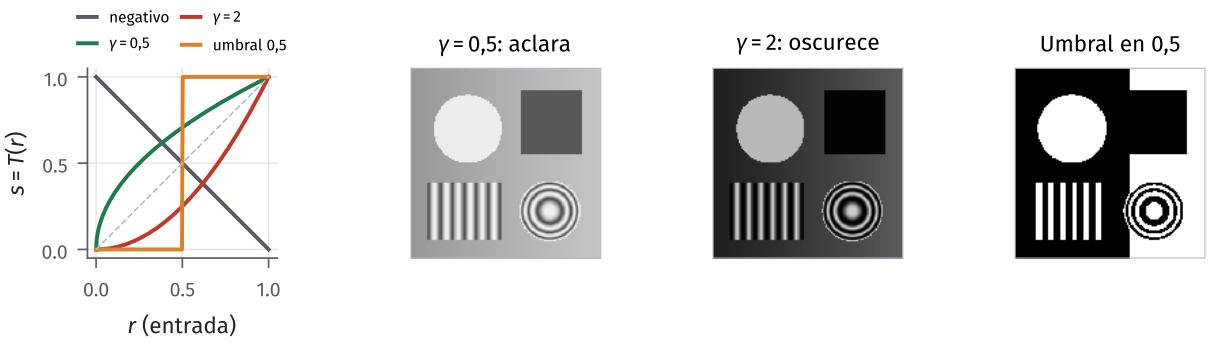


Figura 16. Cuatro transformaciones puntuales y su efecto.

> píxeles de nivel r. Normalizado, $p[r] = h[r]/(MN)$ es la proporción de píxeles de cada nivel, y
>
> $$F(r) = \sum_{i=0}^{r} p[i]$$
>
> es la proporción de píxeles con nivel menor o igual que r.

$p[r]$ y $F(r)$ son la función de probabilidad y la función de distribución del nivel de un píxel elegido al azar (Tema 6). Un histograma concentrado a la izquierda corresponde a una imagen oscura; a la derecha, a una clara; estrecho, a una con poco contraste. El histograma no dice dónde están los píxeles: dos imágenes muy distintas pueden tener el mismo. Si el objeto y el fondo tienen niveles distintos, el histograma tiene dos picos, y un umbral en el valle que los separa segmenta la imagen; el método de Otsu elige ese umbral automáticamente.

La *ecualización* busca repartir los niveles por todo el rango con la transformación

$$s = T(r) = \text{round}\big((L-1)\,F(r)\big),$$

que es creciente, de modo que conserva el orden de los niveles. Si r fuera una variable continua con función de distribución F, la variable $F(r)$ sería uniforme en $[0, 1]$: es el mismo resultado que se usa en el Tema 6 para generar números aleatorios con una distribución dada. Con niveles discretos el histograma resultante no es plano (los píxeles de un mismo nivel van todos al mismo nivel nuevo), pero ocupa todo el rango (figura 17).

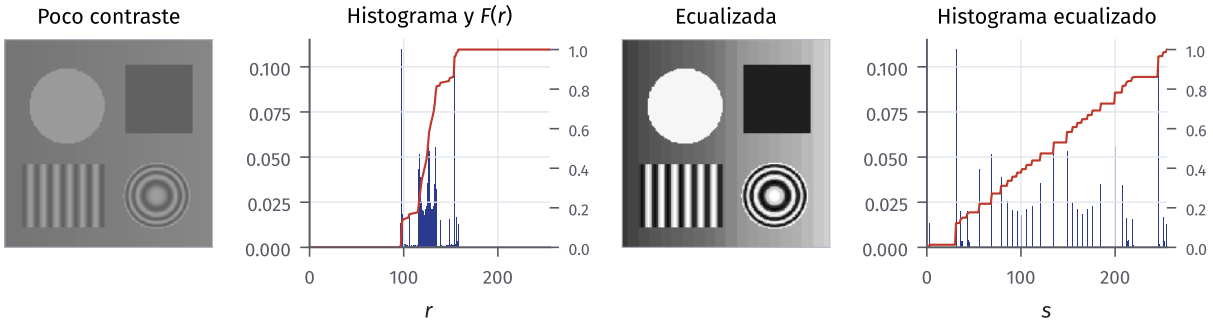


Figura 17. Imagen con poco contraste, su histograma con $F(r)$ en rojo, la imagen ecualizada y su histograma.

En código

Ecualización hecha a mano. Con la Image Processing Toolbox de Matlab, `imhist` dibuja el histograma e `histeq` ecualiza; `histeq` también puede llevar el histograma a una forma prefijada.

Python

```python
import numpy as np
from imagen_prueba import imagen_prueba

f = imagen_prueba()
g = np.round(255*(0.35 + 0.3*f))
g = g.astype(int)           # contraste bajo
h = np.bincount(g.ravel(), minlength=256)
F = np.cumsum(h)/g.size       # distribución
T = np.round(255*F).astype(int)  # s = T(r)
ge = T[g]
print(g.min(), g.max())          # 97 158
print(ge.min(), ge.max())        # 3 255
print(np.std(g), np.std(ge))     # 16.7 73.3
```

Matlab

```matlab
f = imagen_prueba();
g = round(255*(0.35 + 0.3*f)); % 97..158
h = accumarray(g(:) + 1, 1, [256 1]);
F = cumsum(h)/numel(g);       % distribución
T = round(255*F);                % s = T(r)
ge = T(g + 1);
disp([min(g(:)) max(g(:))])      % 97 158
disp([min(ge(:)) max(ge(:))])    % 3 255
disp([std(g(:)) std(ge(:))])     % 16.7 73.3
% Con Image Processing Toolbox: histeq
```

3.4 Convolución bidimensional

Definición 3.2: Convolución 2D

$$g[m,n] = (h * f)[m,n] = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} h[i,j]\, f[m-i, n-j].$$

Es la extensión directa de la convolución 1D, y como ella da la salida de un sistema lineal e invariante al desplazamiento cuya respuesta al impulso $\delta[m,n]$ es $h[m,n]$. En imágenes h es casi siempre un *núcleo* pequeño (3×3, 5×5), es decir, un filtro FIR 2D, y cada píxel de salida es una combinación lineal de sus vecinos.

Si la imagen mide $M\times N$ y el núcleo $K\times K$, la convolución completa mide $(M+K-1)\times(N+K-1)$. Lo habitual es quedarse con la parte central del tamaño de la imagen (opción `'same'`) y, para los píxeles del borde, suponer algo sobre lo que hay fuera: ceros, el valor del borde repetido o la imagen reflejada. Rellenar con ceros oscurece el marco de la imagen.

El núcleo es *separable* si $h[i,j] = u[i]\,v[j]$. Entonces basta filtrar cada fila con v y después cada columna con u, con $2K$ productos por píxel en lugar de K^2: para $K = 15$, 30 en vez de 225. El gaussiano y el de Sobel son separables.

Convolución y correlación

`conv2` de Matlab y `scipy.signal.convolve2d` calculan la convolución; `filter2` e `imfilter` de Matlab calculan la *correlación*, $\sum_{i,j} h[i,j]\,f[m+i,n+j]$, que equivale a convolucionar con el núcleo girado 180°. Para núcleos simétricos dan lo mismo; para el de Sobel, el signo cambia.

Erratas de versiones anteriores

La definición antigua de la convolución 1D escribía $(x * y)(m)$ en lugar de $(x * h)[m]$, y la de la 2D tenía un paréntesis de más y quedaba sin completar.

Núcleos habituales. (Figura 18)

- *Media* de $K \times K$: $h = \frac{1}{K^2}$ en todo el núcleo. Paso bajo: reduce el ruido y emborrona los bordes. Su respuesta en frecuencia es un sinc con lóbulos secundarios.
- *Gaussiano*: $h[i,j] \propto \mathrm{e}^{-(i^2+j^2)/(2\sigma^2)}$, normalizado para sumar 1. Paso bajo sin lóbulos ni artefactos; el más usado para suavizar.
- *Sobel*:

$$S_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -1 \end{bmatrix},$$

  una derivada horizontal ($f[m,n+1] - f[m,n-1]$, al convolucionar) combinada con un suavizado vertical; su traspuesta deriva en vertical. El módulo del gradiente $\sqrt{g_x^2 + g_y^2}$ marca los bordes en cualquier dirección.
- *Laplaciano*: $\left[\begin{smallmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{smallmatrix}\right]$, aproximación de $\partial^2 f/\partial m^2 + \partial^2 f/\partial n^2$. Paso alto: realza los detalles y también el ruido. Sumar a la imagen un múltiplo de $-\nabla^2 f$ la hace más nítida.

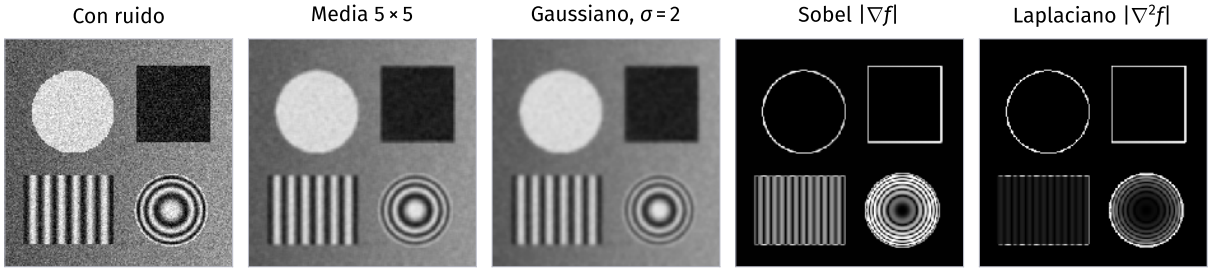


Figura 18. Imagen con ruido suavizada con una media y con un gaussiano, y bordes de la imagen sin ruido detectados con Sobel y con el laplaciano.

En código

Gradiente de Sobel. En el degradado la imagen crece 0,25/256 por columna y g_x vale ocho veces eso: el núcleo de Sobel da la derivada multiplicada por 2×4.

Python

```python
import numpy as np
from scipy.signal import convolve2d
from imagen_prueba import imagen_prueba

f = imagen_prueba()
sx = np.array([[1, 0, -1],
               [2, 0, -2],
               [1, 0, -1]])      # d/dn
media = np.ones((5, 5))/25
gx = convolve2d(f, sx, mode="same")
gy = convolve2d(f, sx.T, mode="same")
g = np.hypot(gx, gy)           # |gradiente|
fs = convolve2d(f, media, mode="same")
bordes = g > 0.5
print(bordes.mean())             # 0.18
print(f[80, 10:20])              # degradado
print(gx[80, 10:20])            # 8*0.25/256
```

Matlab

```matlab
f = imagen_prueba();
sx = [1 0 -1;
      2 0 -2;
      1 0 -1];                    % d/dn
media = ones(5)/25;
gx = conv2(f, sx, 'same');
gy = conv2(f, sx', 'same');
g = hypot(gx, gy);              % |gradiente|
fs = conv2(f, media, 'same');
bordes = g > 0.5;
disp(mean(bordes(:)))             % 0.18
disp(f(81, 11:20))                % degradado
disp(gx(81, 11:20))              % 8*0.25/256
```

3.5 DFT bidimensional

Definición 3.3: DFT 2D

La DFT de una imagen $f[m,n]$ de $M \times N$ y su inversa son

$$F[k,l] = \sum_{m=0}^{M-1}\sum_{n=0}^{N-1} f[m,n]\, e^{-j2\pi\left(\frac{km}{M}+\frac{ln}{N}\right)}, \qquad f[m,n] = \frac{1}{MN}\sum_{k=0}^{M-1}\sum_{l=0}^{N-1} F[k,l]\, e^{j2\pi\left(\frac{km}{M}+\frac{ln}{N}\right)}.$$

La DFT 2D descompone la imagen en *ondas planas* $e^{j2\pi(km/M+ln/N)}$: franjas que tienen k ciclos a lo largo de las filas y l a lo largo de las columnas, orientadas perpendicularmente al vector (k,l). $F[0,0]$ es la suma de todos los píxeles, y $F[0,0]/(MN)$ el nivel medio.

Como $e^{-j2\pi(km/M+ln/N)} = e^{-j2\pi km/M}\, e^{-j2\pi ln/N}$, la DFT 2D es *separable*:

$$F[k,l] = \sum_{m=0}^{M-1}\left[\sum_{n=0}^{N-1} f[m,n]\, e^{-j2\pi ln/N}\right] e^{-j2\pi km/M} \ :$$

se calcula la DFT 1D de cada fila y después la de cada columna del resultado. Con la FFT cuesta $O(MN\log_2(MN))$. En Matlab y en Python, `fft2` e `ifft2`.

Para dibujarla se lleva el origen al centro con `fftshift` (las frecuencias $k \geq M/2$ son en realidad negativas) y se representa $\log(1+|F|)$, porque el valor en el origen es varios órdenes de magnitud mayor que el resto. La figura 19 muestra algunos casos: un coseno da dos puntos simétricos en la dirección perpendicular a sus franjas, tanto más alejados del centro cuanto más finas son, y un cuadrado da el producto de dos sinc, con los lóbulos en cruz. En una imagen real (figura 20) la energía se concentra en las bajas frecuencias, y los bordes rectos producen rayas perpendiculares a ellos.

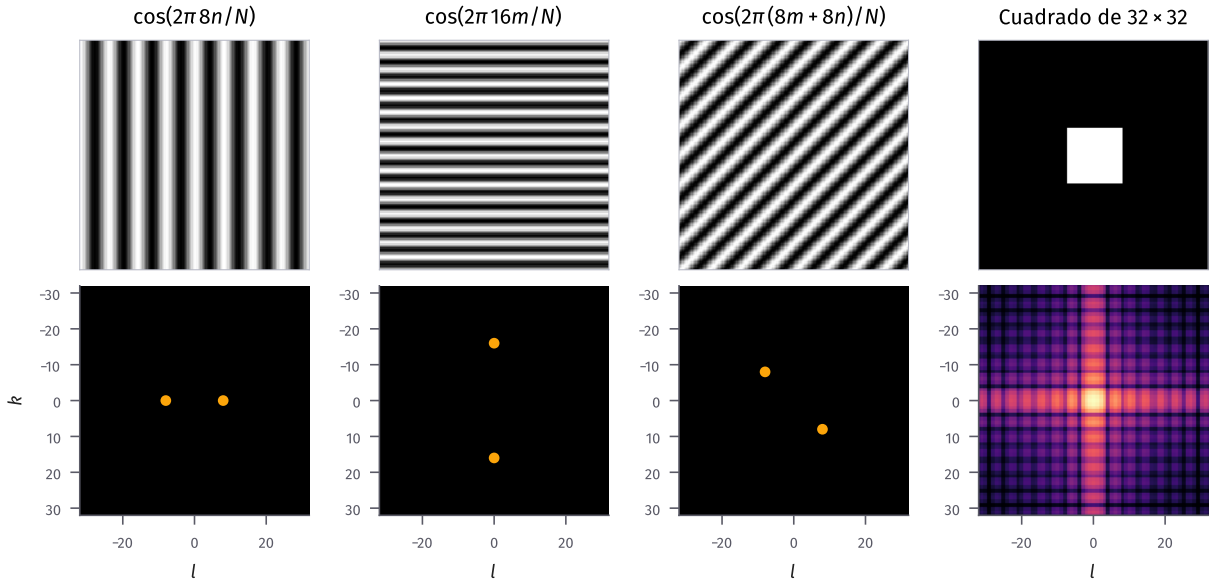


Figura 19. Cuatro imágenes de 128 × 128 y el módulo de su DFT centrada (zona central de ±32).

Propiedad 3.4: Propiedades de la DFT 2D

- *Periodicidad*: $F[k+M,l] = F[k,l+N] = F[k,l]$, y lo mismo f al invertir.
- *Simetría*: si f es real, $F[-k,-l] = F^*[k,l]$ (índices módulo M y N), y $|F|$ es simétrico respecto al origen.

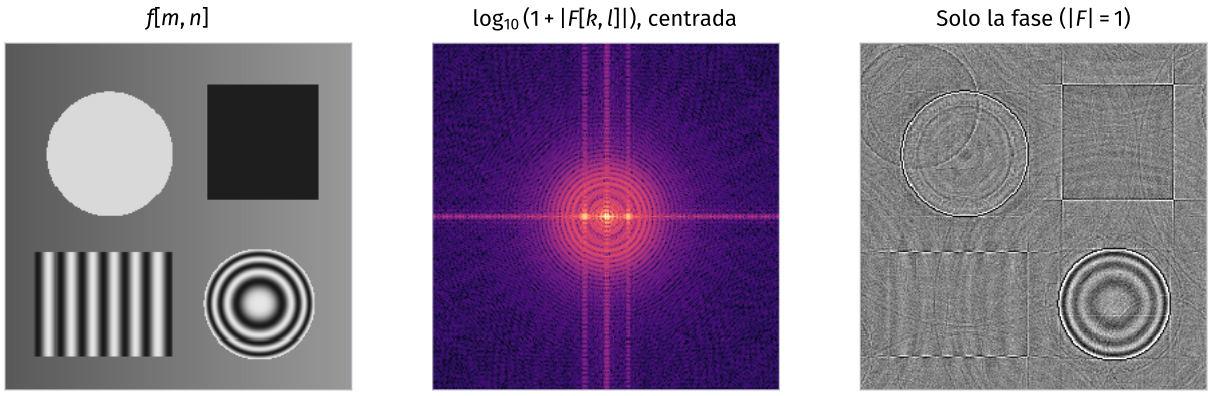


Figura 20. La imagen de prueba, el módulo de su DFT y la imagen que se obtiene conservando solo la fase.

- *Desplazamiento circular*: $f[(m-m_0)_M,(n-n_0)_N] \overset{\mathcal{F}}{\longleftrightarrow} \mathrm{e}^{-j2\pi\left(\frac{km_0}{M}+\frac{ln_0}{N}\right)}F[k,l]$. Mover la imagen no cambia $|F|$.
- *Rotación*: si se gira la imagen un ángulo θ, su transformada (continua) gira el mismo ángulo.
- *Convolución circular*: la convolución circular 2D de dos imágenes de $M\times N$ tiene por DFT el producto de sus DFT.
- *Parseval*: $\sum_{m,n}|f[m,n]|^2 = \frac{1}{MN}\sum_{k,l}|F[k,l]|^2$.

La periodicidad y la simetría son las que permiten centrar la DFT. La fase es tan importante como en 1D: en la figura 20, la imagen reconstruida con $|F|=1$ y la fase original conserva los bordes de los objetos, mientras que con el módulo y una fase cualquiera no se reconoce nada.

Erratas de versiones anteriores

La propiedad de desplazamiento aparecía como $\mathrm{e}^{-j2\pi(u\,m_0+v\,n_0)}F(u,v)$, sin dividir por M y N. Se incluía una propiedad de escalado, $\mathrm{DFT}\{f(am,bn)\} = \frac{1}{ab}F(u/a,v/b)$, que es la de la transformada continua y no tiene sentido para la DFT. La separabilidad se escribía con factores $1/M$ y $1/N$ que no estaban en la definición, y la inversa como «IDFT($f(u,v)$)» en lugar de IDFT$\{F\}$. Además, la DFT 1D se presentaba como «FFT», que es el algoritmo que la calcula.

3.6 Filtrado en el dominio de la frecuencia

Por el teorema de convolución, un filtro se puede aplicar multiplicando: $G[k,l] = H[k,l]\,F[k,l]$ y $g = \mathrm{IDFT}\{G\}$. Es la opción eficiente cuando el núcleo es grande. Dos precauciones: el producto de DFT corresponde a una convolución *circular*, así que para obtener la lineal hay que rellenar con ceros hasta al menos $(M+K-1)\times(N+K-1)$; y para que g sea real, H debe tener la simetría $H[-k,-l] = H^*[k,l]$.

Con $D[k,l] = \sqrt{k^2+l^2}$ la distancia al origen (con las frecuencias negativas bien colocadas):

- *Paso bajo ideal*: $H=1$ si $D\le D_0$ y 0 si no. Su respuesta al impulso es la versión 2D de un sinc, y produce *anillos* alrededor de los bordes (el fenómeno de Gibbs del Tema 3).
- *Paso bajo gaussiano*: $H = \mathrm{e}^{-D^2/(2D_0^2)}$. Su respuesta al impulso también es gaussiana, sin lóbulos: suaviza sin anillos. Corresponde a un gaussiano espacial de $\sigma = N/(2\pi D_0)$ píxeles (ejercicio 16).

- *Paso alto*: $H_{\text{PA}} = 1 - H_{\text{PB}}$; deja los bordes.

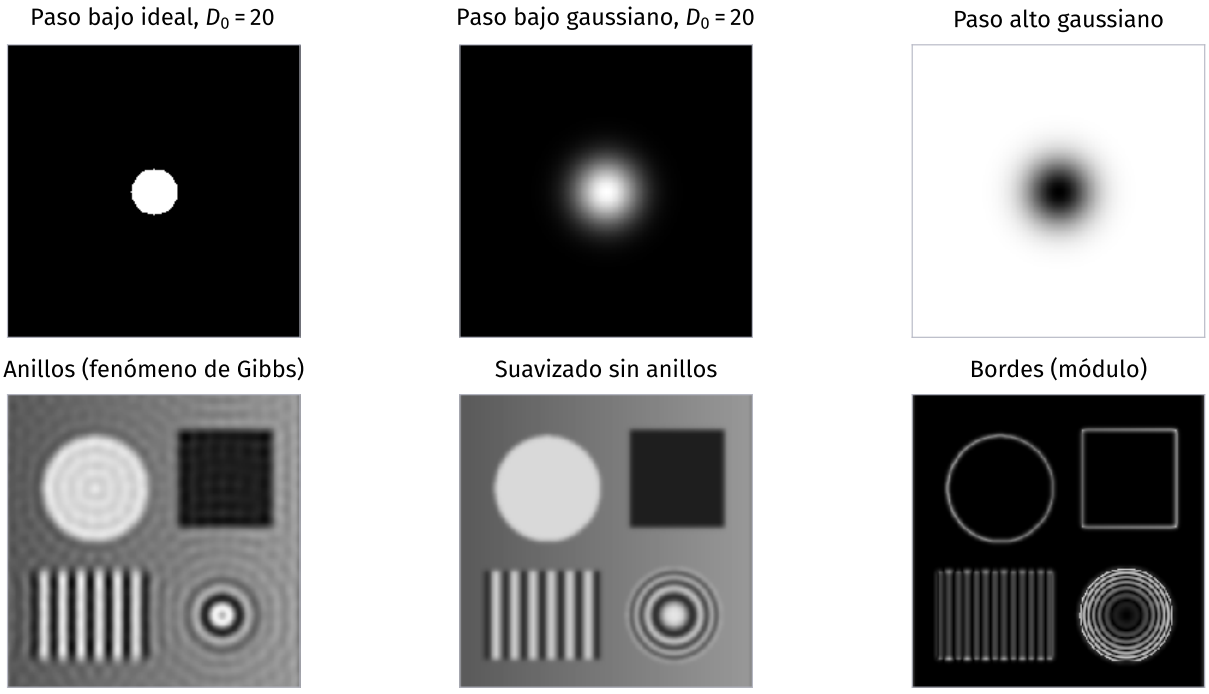


Figura 21. Respuestas en frecuencia (centradas) y resultado de tres filtros aplicados con la DFT.

En código

Paso bajo gaussiano con $D_0 = 20$ aplicado con la DFT. $H[0,0] = 1$, así que el nivel medio no cambia; el filtro conserva el 96 % de la energía.

Python

```python
import numpy as np
from imagen_prueba import imagen_prueba

f = imagen_prueba()
M, N = f.shape
F = np.fft.fft2(f)
k = np.fft.fftfreq(M)*M          # 0..-1
l = np.fft.fftfreq(N)*N
K, L = np.meshgrid(k, l, indexing="ij")
D2 = K**2 + L**2
D0 = 20
H = np.exp(-D2/(2*D0**2))        # gaussiano
g = np.real(np.fft.ifft2(H*F))
E = np.abs(F)**2
print(np.sum(E*H**2)/np.sum(E))  # 0.958
print(f.mean(), g.mean())        # H[0,0]=1
```

Matlab

```matlab
f = imagen_prueba();
[M, N] = size(f);
F = fft2(f);
k = [0:M/2-1, -M/2:-1];          % 0..-1
l = [0:N/2-1, -N/2:-1];
[L, K] = meshgrid(l, k);
D2 = K.^2 + L.^2;
D0 = 20;
H = exp(-D2/(2*D0^2));           % gaussiano
g = real(ifft2(H.*F));
E = abs(F).^2;
disp(sum(E(:).*H(:).^2)/sum(E(:)))
disp([mean(f(:)) mean(g(:))])    % H(1,1)=1
```

3.7 Otras aplicaciones

- *Restauración*: si se conoce el desenfoque H (por movimiento de la cámara, por ejemplo), se puede intentar deshacerlo dividiendo por H; donde $H \approx 0$ esa división amplifica el ruido, y se usa el filtro de Wiener, que tiene en cuenta el espectro del ruido (Tema 6).

- *Compresión*: JPEG divide la imagen en bloques de 8×8, aplica a cada uno la transformada discreta del coseno (DCT) y cuantifica con más fuerza las frecuencias altas; JPEG 2000 usa una DWT 2D.
- *Segmentación y morfología*: umbralizar, detectar bordes, rellenar huecos y medir objetos.
- *Visión por computador*: las redes neuronales convolucionales encadenan muchas convoluciones 2D cuyos núcleos se aprenden a partir de ejemplos.

Ejercicios 11–16.

Bibliografía

- A. V. Oppenheim y R. W. Schafer, *Tratamiento de señales en tiempo discreto*, 3.ª ed. Pearson, 2011. Capítulo 10 (análisis de Fourier de señales con la DFT, incluida la transformada de Fourier dependiente del tiempo).
- J. G. Proakis y D. G. Manolakis, *Tratamiento digital de señales*, 4.ª ed. Pearson Prentice Hall, 2007. Capítulos de la DFT y de bancos de filtros.
- S. Mallat, *A Wavelet Tour of Signal Processing: The Sparse Way*, 3.ª ed. Academic Press, 2009. Referencia completa sobre wavelets, bancos de filtros y eliminación de ruido.
- R. C. Gonzalez y R. E. Woods, *Digital Image Processing*, 4.ª ed. Pearson, 2018. Histograma, filtrado espacial, DFT 2D y filtrado en frecuencia, wavelets en imágenes.
- F. T. Ulaby y A. E. Yagle, *Signals and Systems: Theory and Applications*, 2.ª ed. Michigan Publishing Services, 2024. Capítulos de procesamiento de imágenes y wavelets. Descarga libre en `https://services.publishing.umich.edu/Books/S/Signals-and-Systems`.